

ArrangerAI

Autonomous MIDI Orchestration & Metadata Copilot

Master Architecture & Implementation Specification

Context for AI Code Generator: This document serves as the master specification for building a full-stack, local-first web application. The application processes MIDI files using an agentic swarm, features a highly interactive 3D/animated frontend, and utilizes an asynchronous FastAPI backend. Please build the application incrementally following the phases outlined below.

1. System Architecture & Tech Stack

To achieve Blender-like custom animations in the browser alongside heavy machine learning workloads locally, the stack is divided into a high-performance visual frontend and an asynchronous Python backend.

Frontend (The Visual Experience)

- **Framework:** `Next.js (React)` with `TypeScript`.
- **Styling:** `TailwindCSS` for layout.
- **3D & Animations:** `React Three Fiber (Three.js)`. This is critical for the "Blender-like" requirement. We will use a 3D particle system or floating abstract geometries that react to the music/processing state.
- **UI Transitions:** `Framer Motion` for buttery-smooth page transitions and modal reveals.

Backend (The Engine)

- **API Layer:** `FastAPI`. Built for high concurrency and async operations.
- **Task Queue:** `Redis` + `Celery`. MIDI processing and DL generation take time. The API must return a 202 Accepted and let Celery handle the heavy lifting.

- **Agent Framework:** `LangGraph`. Manages the state between the Theorist, Orchestrator, and Tagging agents.
- **Music/ML Libraries:** `pretty_midi`, `music21`, and `PyTorch` for custom MIDI sequence generation.

2. Implementation Plan (Phase-by-Phase)

Phase 1: The Asynchronous Backbone (Local Setup)

Before building agents or beautiful UI, the asynchronous plumbing must work on the local machine.

- Set up a local **Redis** server via Docker.
- Initialize a FastAPI project with three core endpoints:
 1. `POST /api/v1/session/upload` (Accepts standard MIDI files).
 2. `GET /api/v1/session/status/{task_id}` (Polls Celery for agent state).
 3. `GET /api/v1/session/download/{task_id}` (Returns the final zip package).
- Implement a dummy Celery task that simply sleeps for 10 seconds and updates its status to "Processing", then "Complete", to test the frontend polling mechanism.

Phase 2: The Agentic Swarm (LangGraph Core)

Implement the state machine where the output of one agent dictates the prompt/action of the next.

StateGraph Definition:

- Node 1: `feature_extractor` (Uses `pretty_midi` to find BPM, Key, Chords)
- Node 2: `orchestrator_model` (Passes chord progression to a PyTorch LSTM/Transformer to generate notes)
- Node 3: `metadata_tagger` (Passes BPM, Key, and structural notes to an LLM for A&R sync tags)
- Edge: 1 -> 2 -> 3 -> End

Phase 3: The 3D Frontend Canvas

Build the webapp interface. The key to the "Blender-like" aesthetic is a dark, minimalist UI overlaying a dynamic WebGL canvas.

- Implement a **React Three Fiber** scene in the background. Example: A slow-rotating wireframe mesh or a particle field.

- When a file is uploaded, map the file's file size or name to parameters in the 3D scene (e.g., particles speed up).
- During the "Polling" phase (while Celery is working), trigger a specific animation loop (e.g., glowing nodes connecting, simulating the agent swarm thinking).
- Use **Framer Motion** for the drag-and-drop upload zone, ensuring it scales and glows interactively.

3. Local Device Execution Strategy

To run this smoothly on a local machine, we will containerize the infrastructure dependencies while keeping the frontend and backend running natively for hot-reloading.

- **Docker Compose:** Create a `docker-compose.yml` that spins up Redis (for the message broker).
- **Terminal 1 (Backend):** `uvicorn app.main:app --reload`
- **Terminal 2 (Worker):** `celery -A app.worker worker --loglevel=info`
- **Terminal 3 (Frontend):** `npm run dev`

4. Deployment Possibilities

Once the local project is ready to be shared, the architecture naturally supports horizontal scaling.

- **The Frontend:** Deploy the Next.js application to **Vercel** or **Netlify**. Since the UI is decoupled, this can be served globally via CDN.
- **The Backend Engine:** Package the FastAPI app and Celery worker into Docker containers. Deploy these to **Google Cloud Run** or **AWS ECS**. Cloud Run is excellent because it scales down to zero when you aren't using it, saving costs.
- **The Task Queue:** Use a managed Redis instance (like Upstash or AWS ElastiCache).